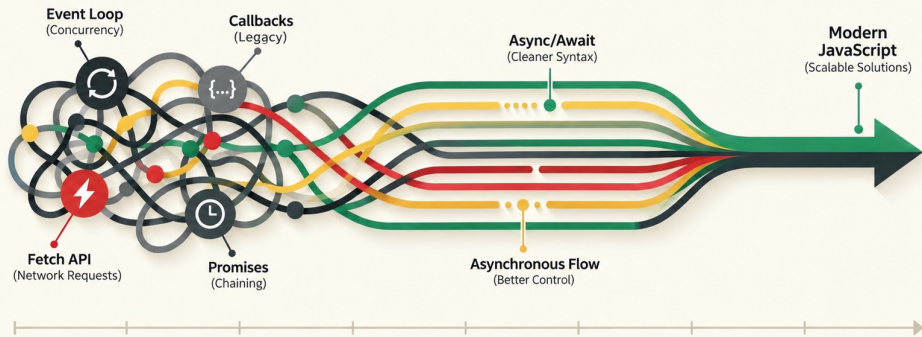


Advanced JavaScript II

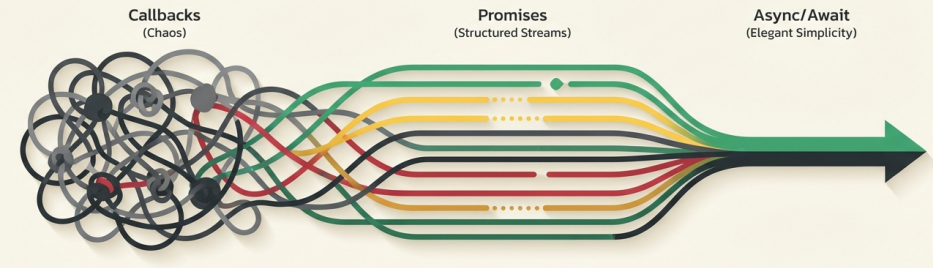
Asynchronous JavaScript. Real-World Power.



© NotebookLM

Mastering Asynchronous JavaScript

วิวัฒนาการจาก Callbacks สู่ Async/Await



© NotebookLM

ธรรมชาติของ JavaScript: ทำทีละอย่าง แต่ไม่เคยหยุดรอ

JavaScript เป็นภาษาที่มีกฎเหล็ก 2 ข้อ:

- **Single-thread:** ทำงานทีละคำสั่ง (เหมือนถนนเลนเดียว)
- **Non-Blocking (Asynchronous ได้):** ถ้าเจองานที่ใช้เวลานาน จะไม่หยุดรอ แต่จะส่งไปทำเบื้องหลัง

```
console.log("Start");
setTimeout(() => {
  console.log("Async Task");
}, 2000);
console.log("End");
```



© NotebookLM

Synchronous vs Asynchronous

🔴 Synchronous (Blocking)

ทำงานเรียงตามลำดับ
ทุกอย่างรอจนเสร็จ

```
console.log('เริ่ม')
console.log('ดาวน์โหลด')
console.log('จบ')
```

Output Box
เริ่ม -> ดาวน์โหลด -> จบ

🟢 Asynchronous (Non-Blocking)

งานที่ใช้เวลานานจะถูกส่งไปทำเบื้องหลัง
(ไม่ล็อกโปรแกรม)

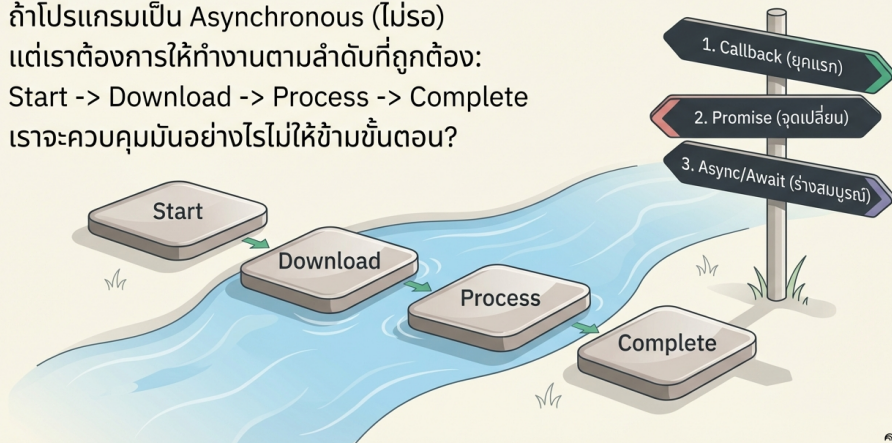
```
console.log('เริ่ม')
setTimeout(() => { console
  .log('ดาวน์โหลด') }, 3000)
console.log('จบ')
```

Output Box
เริ่ม -> จบ -> ดาวน์โหลด
(ดาวน์โหลดไปล่าช้า)

© NotebookLM

ปัญหาใหญ่: เราจะควบคุมลำดับการทำงานได้อย่างไร?

ถ้าโปรแกรมเป็น Asynchronous (ไม่รอ)
แต่เราต้องการให้ทำงานตามลำดับที่ถูกต้อง:
Start -> Download -> Process -> Complete
เราจะควบคุมมันอย่างไรไม่ให้ข้ามขั้นตอน?



Generation 1: Callback ฟังก์ชันที่เรียกเมื่อพร้อม

Callback คือ ฟังก์ชันที่ส่งเข้าไปเป็นพารามิเตอร์ และจะถูกเรียกเมื่อทำงานเสร็จ (เช่น จำลองการคำนวณที่ใช้เวลา 3 วินาที)

```
function calculate(x, y, callback) {  
  console.log('กำลังคำนวณ...')  
  setTimeout(() => {  
    const sum = x + y  
    callback(sum)  
  }, 3000)  
}  
  
function display(result) {  
  console.log('ผลบวก = ${result}')  
}  
  
calculate(100, 50, display)
```

<-- เรียกใช้งานเมื่อเสร็จ

วิกฤตการณ์ Callback Hell 🔥

```
download(()=>{  
  process(()=>{  
    upload(()=>{  
      finish(()=>{  
        ...  
      })  
    })  
  })  
})
```

การนำฟังก์ชันซ้อนกันไปเรื่อยๆ เพื่อเช็คลำดับการทำงาน ทำให้เกิดปัญหา:

- ❌ โค้ดเยื้องลึกเป็นรูปสามเหลี่ยม
- ❌ อ่านยาก และสับสน
- ❌ Debug หาข้อผิดพลาดได้ยากมาก

Generation 2: Promise ทางออกแห่งความสับสน

Promise คือวัตถุ (Object) สำหรับจัดการงาน Asynchronous เพื่อหลีกเลี่ยงการซ้อนทับกันของ Callback

```
function downloading(url){  
  return new Promise(function(resolve, reject){  
    console.log(`กำลังโหลดไฟล์จาก ${url}`)  
    setTimeout(() => {  
      if(connect){  
        resolve(`โหลด ${url} เรียบร้อย`)  
      } else {  
        reject('เกิดข้อผิดพลาด')  
      }  
    }, 3000)  
  })  
}
```

กายวิภาคของ Promise (3 สถานะหลัก)



ไฟเหลือง (Pending): กำลังทำ / กำลังรอข้อมูล
(เหมือนกำลังโหลดไฟล์)

ไฟเขียว (Fulfilled / Resolved):
สำเร็จ (ได้ข้อมูลครบถ้วน ส่งต่อเข้า .then())

ไฟแดง (Rejected):
ผิดพลาด (เกิด error ส่งต่อเข้า .catch())

© NotebookLM

หลุดพ้นจาก Hell ด้วย .then() และ .catch()

```
download(url1)
  .then(function(result){
    console.log(result)
    return download(url2) // สำคัญมาก 🔥
  })
  .then(function(result){
    console.log(result)
    return download(url3) // สำคัญมาก 🔥
  })
  .catch(function(err){
    console.log(err)
  })
```

- ✅ อ่านง่ายกว่า Callback ไม่เยื้องลึก
- ✅ ควบคุมลำดับอย่างเป็นระเบียบด้วย then()
- ✅ ดักจับ Error ที่เดียวด้วย catch()

© NotebookLM

Generation 3: Async / Await สร้างสมบูรณ์ที่อ่านง่ายที่สุด

Async/Await คือ Syntax สมัยใหม่ที่ทำงานอยู่บนพื้นฐานของ Promise แต่นำเสนอในรูปแบบที่ต่างออกไป:

- ✨ อ่านเหมือน Synchronous: โค้ดเรียงตรงๆ จากบนลงล่าง
- ✨ เข้าใจง่าย: ไม่ต้องมี .then() ต่อท้ายยาวๆ
- ✨ ยังคงเป็น Asynchronous: เบื้องหลังยังคงไม่บล็อกการทำงานของโปรแกรมหลัก



© NotebookLM

พลังของ Async/Await ในการใช้งานจริง

```
async function start(){
  try {
    console.log(await download(url1))
    console.log(await download(url2))
    console.log(await download(url3))
    console.log(await download(url4))
    console.log(await download(url5))
  } catch(err) {
    console.log(err)
  }
}
start()
```

ใช้ await เพื่อรอข้อมูลให้ครบก่อนไปบรรทัดถัดไป โดยเขียนครอบด้วย try/catch เพื่อจัดการ Error อย่างสะอาดตา

© NotebookLM

ลงสู่โลกจริงด้วย API: รู้จักกับ fetch()

fetch() คือฟังก์ชันมาตรฐานสำหรับส่ง HTTP Request ไปหา Server ถูกใช้งานในระบบจริงตลอดเวลา เช่น:



ดึงข้อมูลจาก API



ส่งข้อมูลไป Backend



ระบบ Login



โหลดข้อมูลจากฐานข้อมูล



สำคัญมาก: fetch() คืนค่าเป็น Promise เสมอ!
จึงต้องใช้คู่กับ then() หรือ async/await

© NotebookLM

เปรียบเทียบการใช้งาน fetch() ถึงข้อมูล

แบบ Promise (ดั้งเดิม)

```
fetch('https://api.example.com/data')  
  .then(response => response.json())  
  .then(data => { console.log(data) })  
  .catch(error => { console.log(error) })
```

★ แบบ Async/Await (นิยมที่สุด)

```
async function loadData(){  
  try {  
    const response = await fetch  
      ('https://api.example.com/data')  
    const data = await response.json()  
    console.log(data)  
  } catch(error) {  
    console.log(error)  
  }  
}  
loadData()
```

© NotebookLM

ภาพรวม: การเชื่อมต่อระบบ (Frontend <-> Backend)



Browser (Frontend)
-> เรียกใช้ fetch()
(สถานะ Pending)



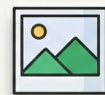
Backend API
-> รับคำขอ



Database
-> ค้นหา
ข้อมูลภาพสินค้า



Response
-> ส่งข้อมูลกลับมา



แสดงผล
-> Frontend ใช้ await
รอจนได้ข้อมูลครบ
แล้วจึงแสดงภาพบนเว็บ

ทั้งหมดนี้ทำงานด้วยแนวคิด Single Thread + Event Loop + Non-Blocking ภายใต้ Promise-based Asynchronous

© NotebookLM